Министерство образования Республики Беларусь
Учреждение образования «Белорусский государственный университет информатики и радиоэлектроники»

Факультет компьютерных систем и сетей
Кафедра информатики
Дисциплина «Архитектура вычислительных систем»

ОТЧЕТ
к лабораторной работе №1-3
на тему:
«ПРОГРАММИРОВАНИЕ АРИФМЕТИЧЕСКОГО СОПРОЦЕССОРА»
БГУИР 6-05-0612-02 113

Выполнил студент группы 453503
ХАЛАМОВ Николай Андреевич

(дата, подпись студента)

Проверил ассистент каф. информатики
КАЛИНОВСКАЯ Анастасия Александровна

(дата, подпись преподавателя)

Минск 2026

1 ИНДИВИДУАЛЬНОЕ ЗАДАНИЕ

Задание 1

1 Осуществить методом математического выполнения функции согласно варианту задания. Значение аргумента x изменяется от a до b с шагом h. Для каждого x найти значения функции Y(x), суммы S(x) и число итераций n, при котором достигается требуемая точность ε = |Y(x)-S(x)|. Результат вывести в виде таблицы. Значения a, b, h и ε вводятся с клавиатуры.

2 Осуществить написание программы на любом удобном языке программирования с вставками кода ассемблера для выполнения задачи на i количестве итерации (i>5000) для получения достоверных результатов эксперимента.

3 Результатом будут графики нагрузки на одно ядро процессора и на на все ядра процессора. График нагрузки на GPU. По оси X – время выполнения, по оси Y – количество итераций (i). Intel – два графика на одно ядро, с функцией Hyper-Threading и без.

Задание 2

1 Выбрав вариант задания, а именно, заданную функцию осуществить написание программы на любом удобном языке программирования с вставками кода ассемблера для выполнения задачи на i количестве итерации (i>5000) для получения достоверных результатов эксперимента.

2 Выполнение задачи должно осуществляться в операционной системе без графической оболочки (Linux Ubuntu, Debian и др. – можно в LiveCD режиме, можно посредством установки второй системы на ноутбук).

3 Результатом послужат графики нагрузки на все ядра процессора. По оси X – время выполнения, по оси Y – количество итераций (i). Для Intel – два графика на все ядра, с функцией Hyper-Threading и без.

Задание 3

1 Осуществить выполнение заданной функции с использованием параллельных вычислений на GPU/iGPU в соответствии с выбранным вариантом задания.

2 Напишите программу на любом удобном языке программирования, с использованием CUDA или OpenCL. Выполнение задачи должно осуществляться в операционной системе без графической оболочки . Реализуйте параллельные вычисления, используя возможности GPU/iGPU, чтобы ускорить выполнение задачи. Убедитесь, что ваша программа может обрабатывать более 5000 итераций для получения достоверных результатов.

3 Результатом станут графики нагрузки на одно ядро процессора. По оси X – время выполнения, по оси Y – количество итераций (i).

$$7.\ S(x)=\sum_{k=0}^{n}\frac{\cos(kx)}{k!}, \qquad Y(x)=e^{\cos x}\cos(\sin(x))$$

Рисунок 1 – Заданные функции

2 ВЫПОЛНЕНИЕ РАБОТЫ

Для выполнения задания 1 была разработана программа на языке C++ с использованием ассемблерных вставок для вычисления члена ряда. Ключевым элементом реализации является inline-ассемблер, который позволяет напрямую работать с регистрами. Такой подход обеспечивает максимальную производительность на низком уровне и соответствует требованию работы с сопроцессором.

```
#include <chrono>
#include <cmath>
#include <fstream>
#include <iostream>
#include <mutex>
#include <sched.h>
#include <thread>
#include <condition_variable>
#include <vector>

void pinToCore(int core) {
    cpu_set_t set;
    CPU_ZERO(&set);
    CPU_SET(core, &set);
    sched_setaffinity(0, sizeof(set), &set);
}

double CalculateFunction(double x) {
    return std::exp(std::cos(x)) * std::cos(std::sin(x));
}

double computeSeriesASM(double x, int max_iter, double eps, bool
use_eps, int &out_iters) {
    double series = 1.0;
    double func = CalculateFunction(x);
    double fact = 1.0;
    double term = 1.0;
    double cos_val = 1.0;
    int k = 0;

    do {
        if (k > 0) {
            fact = fact * (double)k;
            double kx = k * x;

            asm volatile (
                "movsd %[kx], %%xmm0\n"
                "call cos\n"
                "movsd %%xmm0, %[cos_val]\n"
                : [cos_val] "=m" (cos_val)
                : [kx] "m" (kx)
                : "xmm0", "rax", "rcx", "rdx", "rsi", "rdi", "memory"
            );

            asm volatile (
                "movsd %[cos_val], %%xmm0\n"
                "divsd %[fact], %%xmm0\n"
                "movsd %%xmm0, %[term]\n"
                : [term] "=m" (term)
                : [cos_val] "m" (cos_val), [fact] "m" (fact)
                : "xmm0"
```

```
                );

                series += term;
            }

            out_iters = k;
            k++;

            if (use_eps && std::abs(series - func) < eps) break;

        } while (k <= max_iter);

        return series;
    }

    void runSingleCoreBenchmark(std::string fileName, double x) {
        std::ofstream fileOut(fileName);
        fileOut << "iterations,time_ns\n";
        pinToCore(0);

        for (int iterations = 1000; iterations <= 50000; iterations += 100)
{
            int dummy_iters = 0;
            auto start = std::chrono::high_resolution_clock::now();
            computeSeriesASM(x, iterations, 0.0, false, dummy_iters);
            auto end = std::chrono::high_resolution_clock::now();

            double duration = std::chrono::duration<double, std::nano>(end -
start).count();
            fileOut << iterations << "," << duration << "\n";
        }
    }

    void runMultiCoreBenchmark(std::string fileName, double x) {
        unsigned int cores = std::thread::hardware_concurrency();
        std::ofstream fileOut(fileName);
        fileOut << "core,iterations,time_ns\n";

        std::vector<std::thread> threads;
        std::mutex file_mtx, start_mtx;
        std::condition_variable start_cv;
        bool start_flag = false;

        for (int i = 0; i < cores; i++) {
            threads.emplace_back([&, i]() {
                pinToCore(i);

                {
                    std::unique_lock<std::mutex> lk(start_mtx);
                    start_cv.wait(lk, [&]() { return start_flag; });
                }

                for (int iterations = 1000; iterations <= 50000; iterations
+= 100) {
                    int dummy = 0;
                    auto start = std::chrono::high_resolution_clock::now();
                    computeSeriesASM(x, iterations, 0.0, false, dummy);
                    auto end = std::chrono::high_resolution_clock::now();

                    double     time     =     std::chrono::duration<double,
std::nano>(end - start).count();
                    std::lock_guard<std::mutex> lock(file_mtx);
                    fileOut << i << "," << iterations << "," << time <<
"\n";
```

```
            }
        });
    }

    start_flag = true;
    start_cv.notify_all();

    for (auto &t : threads) t.join();
}
```

Для исследования влияния Hyper-Threading на производительность однопоточных вычислений выполнялись два запуска программы: с включённой и отключённой функцией. Проверка текущего состояния процессора осуществлялась командой lscpu | grep -E "Thread|Core". При включённом Hyper-Threading параметр "Thread(s) per core" имеет значение 2, при отключённом – 1.

По умолчанию на процессорах Intel функция Hyper-Threading активна, поэтому первый запуск выполнялся в стандартной конфигурации. Для отключения Hyper-Threading использовался программный метод без перезагрузки системы – запись значения "off" в файл /sys/devices/system/cpu/smt/control.

В ходе эксперимента были получены данные о времени выполнения программы в наносекундах для каждого количества итераций. На основе этих данных построены графики зависимости времени выполнения от числа итераций для двух режимов: с Hyper-Threading и без него.

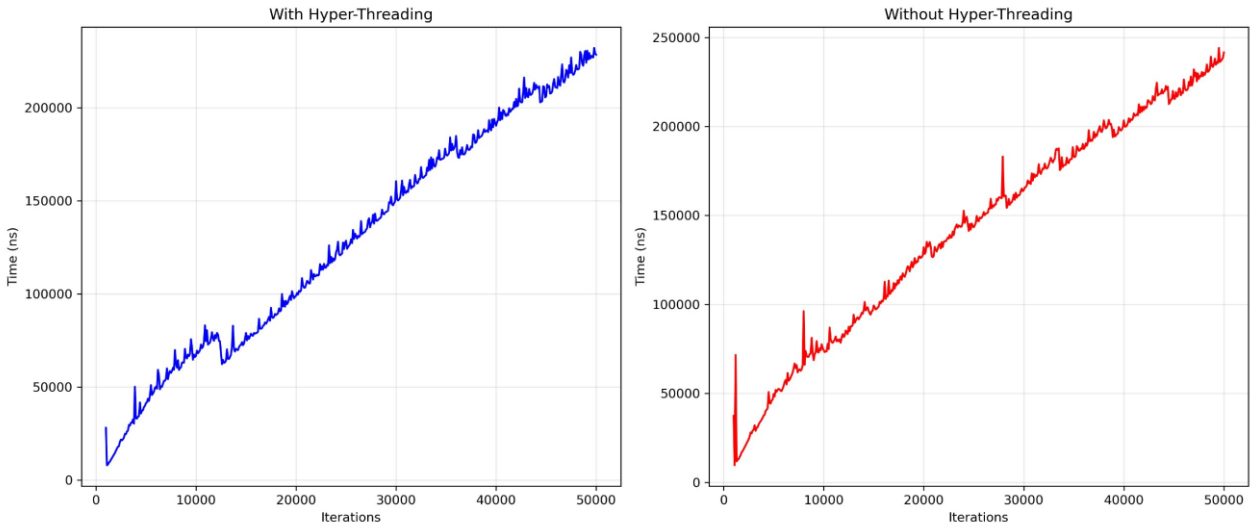


Рисунок 2 – График выполнения в одноядерном режиме с опцией Hyper-Threading и без неё

Анализ графиков показывает, что в режиме одного ядра разница в производительности между включённым и отключённым Hyper-Threading незначительна. Это объясняется следующим образом:

1 Hyper-Threading позволяет одному физическому ядру обрабатывать два потока команд одновременно, но при выполнении одного

вычислительного потока второй логический процессор остаётся незагруженным. Таким образом, потенциальное преимущество не может быть реализовано в полной мере.

2 Вычисления ряда являются последовательной задачей: каждый следующий член ряда зависит от предыдущего. Алгоритм плохо поддаётся параллелизации на уровне одного ядра, поэтому наличие второго логического процессора не даёт существенного выигрыша.

Для выполнения задания 2 существующая программа была расширена функционалом многопоточного выполнения на всех ядрах процессора. Ключевым элементом реализации является создание пула потоков, где каждый поток привязывается к конкретному ядру. Это гарантирует, что каждый вычислительный поток будет выполняться на своём физическом ядре без миграции между ядрами.

Особое внимание уделено синхронизации потоков. Для одновременного старта всех потоков используется std::condition_variable и флаг start_flag. Все потоки ожидают сигнала от главного потока, после чего начинают вычисления одновременно. Это позволяет получить корректные замеры времени при параллельной работе.

Для обеспечения потокобезопасной записи результатов в файл используется std::mutex. Блокировка захватывается перед записью строки в CSV-файл и освобождается после завершения записи. Это предотвращает состояние гонки, при котором несколько потоков одновременно пытаются записать данные в файл, что может привести к перемешиванию строк или повреждению данных.

Для исследования влияния Hyper-Threading на производительность в многоядерном режиме выполнялись два запуска программы: с включённой и отключённой функцией. В многопоточном режиме каждый поток выполняет одинаковый объём вычислений – полный цикл итераций от 1000 до 50000. Однако все потоки работают параллельно на разных ядрах, что позволяет измерить производительность системы при полной загрузке всех вычислительных ресурсов.

Важно отметить, что каждый поток вычисляет сумму ряда независимо от других потоков, так как задача является параллельной. Это означает, что вычисления для разных потоков не требуют обмена данными или синхронизации в процессе работы – достаточно синхронизировать только старт и запись результатов.

Анализ графиков, полученных в ходе эксперимента, показывает разницу между одноядерными и многоядерными вычислениями.

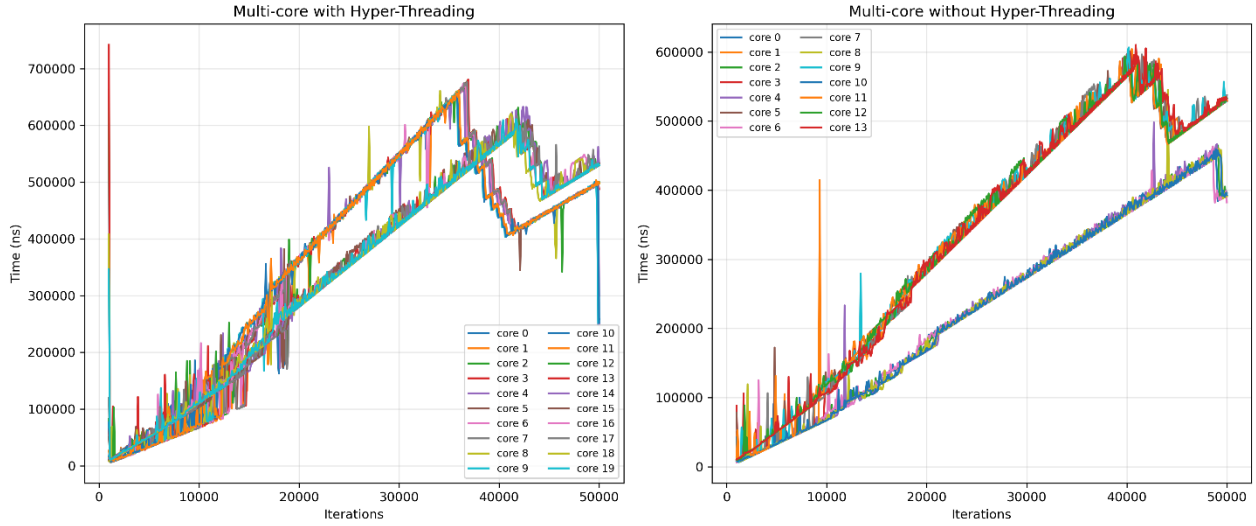


Рисунок 3 – График выполнения в многоядерном режиме с опцией Hyper-Threading и без неё

Режим Hyper-Threading почти не влияет на скорость выполнения. Однако в многоядерном режиме вычисления происходят дольше, чем в одноядерном режиме. Это объяснимо исполльзованием mutex для синхронизации. Накладные расходы на синхронизацию вносят ощутимый вклад в общее время выполнения, особенно при большом количестве операций записи.

Для выполнения задания 3 была разработана программа на языке C++ с использованием технологии OpenCL, которая обеспечивает кроссплатформенную поддержку вычислений на графических процессорах.

Ядро OpenCL реализует вычисление суммы ряда непосредственно на GPU. Ключевым аспектом GPU-вычислений является управление передачей данных между CPU и GPU. Операции записи и чтения буферов выполняются синхронно, что гарантирует завершение передачи перед продолжением выполнения программы, а само ядро выполняется асинхронно.

Важно отметить, что накладные расходы на передачу данных и запуск ядра могут быть значительными, особенно при малом объёме вычислений. Это является определяющим фактором эффективности GPU.

GPU демонстрирует значительно худшую производительность при обработке одной точки. Это объясняется накладными расходами на инициализацию OpenCL, компиляцию ядра, передачу данных и синхронизацию. Для одной задачи эти затраты превышают вычислительную пользу от параллельной архитектуры.

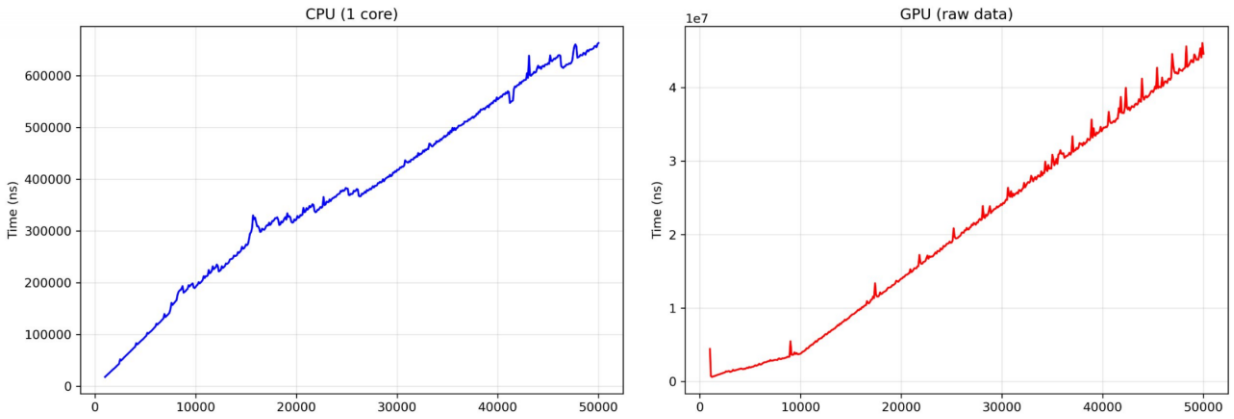


Рисунок 4 – График выполнения на CPU и на GPU

При малом количестве точек CPU остаётся быстрее GPU. Однако с увеличением числа точек GPU начинает демонстрировать существенное ускорение. Полученные результаты объясняются архитектурными особенностями GPU:

1 Массовый параллелизм: Графический процессор Intel содержит 96 вычислительных блоков, каждый из которых способен выполнять несколько потоков одновременно. При обработке тысяч точек GPU может задействовать все вычислительные ресурсы.

2 Накладные расходы: Запуск OpenCL-ядра требует времени на передачу данных, компиляцию и синхронизацию. При малом объёме вычислений эти затраты доминируют.

Это демонстрирует, что встроенная графика Intel может превосходить мощный многоядерный CPU в задачах, хорошо поддающихся параллелизации.

ВЫВОД

В ходе выполнения лабораторных работ было проведено комплексное исследование производительности процессора Intel Core i9. Эксперименты охватили три режима работы: однопоточные вычисления на одном ядре (с технологией Hyper-Threading и без неё), многопоточные вычисления на всех ядрах процессора, а также параллельные вычисления с использованием GPU через OpenCL.

В одноядерном режиме технология Hyper-Threading показала незначительное улучшение производительности, что объясняется последовательным характером вычислений. При выполнении одного вычислительного потока второй логический процессор на том же физическом ядре остаётся практически незагруженным, поэтому потенциальные преимущества SMT не могут быть реализованы.

В многоядерном режиме вычисления происходят дольше, чем в одноядерном режиме. Накладные расходы на синхронизацию увеличивают общее время выполнения, особенно при большом количестве операций записи.

Таким образом, технология Hyper-Threading наиболее эффективна в сценариях с высокой степенью параллелизма и разнородной вычислительной нагрузкой, тогда как для последовательных алгоритмов её влияние минимально.

Показательные результаты получены при сравнении центрального и графического процессоров. При обработке одной точки GPU уступает CPU из-за высоких накладных расходов на передачу данных, компиляцию ядра и синхронизацию. Однако при увеличении объёма вычислений GPU раскрывает свой потенциал. Это подтверждает, что архитектура GPU, ориентированная на массовый параллелизм, эффективна для задач, которые можно разбить на множество независимых подзадач.

Выполненные лабораторные работы позволили на практике изучить особенности работы современных вычислительных систем на разных уровнях параллелизма: от однопоточных вычислений на уровне ассемблерных инструкций до массового параллелизма на графическом процессоре. Полученные результаты подтверждают теоретические положения об эффективности различных вычислительных архитектур и могут быть использованы при выборе оптимальной конфигурации для решения конкретных задач.